Resumo para a Prova A1

Desenvolvimento de Software Visual. Ciência da Computação, 4º semestre, 2026/2. Prof. Diogo Steinke Deconto. Prova A1: **23/11/2026**.

A A1 avalia o "resultado final do processo" (Plano de Ensino), então o foco é **React consumindo a API em C#**, mas a API continua valendo. O conteúdo da A2 está resumido no §12 e completo no `Resumo-Prova-A2.pdf`.

Data	Aula prevista no calendário	Neste resumo
05/10	React: introdução ao React e configuração do ambiente	§1, §2
19/10	React: componentização e estado	§3 a §6
26/10 e 09/11	React: CRUD com a API em C#	§7, §8
16/11	React: validações de formulário	§9

Material preparado **antes** das aulas de React, com base no calendário e no Plano de Ensino (Unidade V: frameworks JavaScript para SPA, layouts responsivos com grids, regras de negócio e validação de formulário). O código completo e testado está em `08-React-Ecommerce`. Quando as aulas acontecerem, confira se o professor usou as mesmas bibliotecas (ex.: `axios` ou `fetch`).

1. SPA, React e o ambiente (Node, npm, Vite)
2. Estrutura do projeto e TypeScript básico
3. Componentes e JSX
4. Props
5. Estado (useState) e eventos
6. Listas, keys e renderização condicional
7. useEffect e consumo da API (axios, async/await, CORS)
8. CRUD completo e rotas (React Router)
9. Formulários e validação
10. Layout responsivo
11. Pegadinhas
12. Revisão relâmpago da API (A2)
13. Questões com gabarito

1. SPA, React e o ambiente

- **SPA** (Single Page Application): o navegador carrega **uma** página HTML só, e o JavaScript troca o conteúdo conforme a navegação, sem recarregar a página. Os dados vêm da API em JSON. Diferente das páginas tradicionais, em que cada clique pede um HTML novo ao servidor.
- **React**: biblioteca JavaScript (da Meta) para construir interfaces a partir de **componentes**. Ela mantém um **DOM virtual** (cópia em memória da tela): quando o estado muda, compara a nova versão com a anterior (*reconciliação*) e atualiza no DOM real só o que mudou.
- **Front-end × back-end**: o React (front) roda no navegador e só exibe e coleta dados. A API C# (back) tem as regras e o banco. Os dois conversam por **HTTP + JSON**.
- **Node.js**: ambiente que roda JavaScript fora do navegador, necessário para as ferramentas de desenvolvimento. **npm**: gerenciador de pacotes do Node (equivale ao NuGet do .NET).
- **Vite**: ferramenta que cria o projeto e roda o servidor de desenvolvimento com recarga instantânea (porta padrão **5173**). Também gera a versão final (`npm run build` → pasta `dist/`).

Comando	O que faz	Equivalente .NET
`npm create vite@latest front -- --template react-ts`	cria o projeto React com TypeScript	`dotnet new web`
`npm install`	baixa as dependências do `package.json` para `node_modules/`	`dotnet restore`
`npm install axios react-router-dom`	adiciona bibliotecas	`dotnet add package`
`npm run dev`	roda em desenvolvimento	`dotnet run` / `watch`
`npm run build`	compila para produção	`dotnet build` / `publish`

> **node_modules**
> Nunca vai para o Git (está no `.gitignore`): é enorme e é recriado com `npm install` a partir do `package.json` (dependências) e do `package-lock.json` (versões exatas). Ao clonar um projeto React, o primeiro passo é `npm install`.

2. Estrutura do projeto e TypeScript básico

```
front/
├── index.html          ← a ÚNICA página. Tem <div id="root"></div>
├── package.json        ← dependências e scripts (dev, build)
├── vite.config.ts
└── src/
    ├── main.tsx        ← ponto de entrada: createRoot(...).render(<App />)
    ├── App.tsx         ← componente raiz (rotas e menu)
    ├── models/         ← interfaces (formato dos dados)
    ├── services/       ← acesso à API
    └── components/     ← componentes .tsx
```

TypeScript = JavaScript + **tipos**, verificados na compilação (como no C#). Arquivo `.ts` é só código e `.tsx` é código com JSX.

```ts
let nome: string = "Ana";
let idade: number = 20;    // int e double são number
let ativo: boolean = true;
let lista: string[] = ["a", "b"];
const PI = 3.14;           // const: não reatribui
function somar(a: number, b: number): number {
  return a + b;
}
const dobro = (x: number) => x * 2; // arrow function
```

```ts
// interface = formato de um objeto (como o model do C#)
export interface Produto {
  id?: string;      // ? = opcional
  nome: string;
  valor: number;
  criadoEm?: string;
}
// JSON da API C# chega em camelCase:
// { "id": "...", "nome": "Mouse", "valor": 100 }
`Template string: ${nome}`  // = $"..." do C#
=== compara valor E tipo (use sempre)
```

3. Componentes e JSX

Componente é uma função que devolve **JSX**. É a unidade de reaproveitamento da tela (botão, formulário, tabela...). A interface vira uma **árvore de componentes**.

```jsx
function Cabecalho() {
  const titulo = "Ecommerce";
  return (
    <header className="topo">          {/* className, não class */}
      <h1>{titulo}</h1>               {/* {} insere expressão JS */}
      <p>Hoje: {new Date().toLocaleDateString()}</p>
      <img src="logo.png" alt="logo" />  {/* tags vazias precisam de / */}
    </header>
  );
}
export default Cabecalho;
// uso em outro componente:  import Cabecalho from "./Cabecalho";   <Cabecalho />
```

Regras do JSX

- O nome do componente começa com **letra maiúscula** (`<Cabecalho />`). Minúscula é tag HTML.
- Devolve **um único elemento raiz**. Para não criar `div` extra, use o **fragment** `<> ... </>`.
- `className` no lugar de `class`, `htmlFor` no lugar de `for`. Atributos em camelCase (`onClick`, `onChange`).
- Toda tag precisa fechar: `<br />`, `<input />`. Comentário: `{/* ... */}`. Estilo inline é objeto: `style={{ color: "red" }}`.

4. Props

Props são os "parâmetros" de um componente: dados que o **pai** passa para o **filho**. São **somente leitura** (o filho não altera). O fluxo de dados no React é **unidirecional**: de cima para baixo.

```jsx
interface CartaoProps { nome: string; valor: number; onRemover?: () => void; }

function CartaoProduto({ nome, valor, onRemover }: CartaoProps) {   // desestruturação
  return (
    <div>
      <h3>{nome}</h3> <p>R$ {valor.toFixed(2)}</p>
      {onRemover && <button onClick={onRemover}>Remover</button>}
    </div>
  );
}
// Pai:
<CartaoProduto nome="Mouse" valor={100} onRemover={() => remover(id)} />
```

Para o filho "avisar" o pai (de baixo para cima), o pai passa uma **função** como prop (`onRemover`) e o filho a chama. Strings vão entre aspas e qualquer outro valor vai entre `{}`.

5. Estado (useState) e eventos

Estado é um dado do componente que, quando muda, faz o React **re-renderizar** (desenhar de novo) o componente. Uma variável comum não serve: o valor muda, mas a tela não.

```tsx
import { useState } from "react";

function Contador() {
  const [contador, setContador] = useState<number>(0);   // [valor atual, função que altera] = useState(inicial)
  return (
    <div>
      <h3>{contador}</h3>
      <button onClick={() => setContador(contador + 1)}>+1</button>
      <button onClick={() => setContador(0)}>Zerar</button>
    </div>
  );
}
```

Regras de ouro do estado

- **Nunca** altere direto (`contador = 5` ou `produtos.push(p)`). Sempre use o `set...`.
- Para listas e objetos, crie um **novo**: `setProdutos([...produtos, novo])`, `setProdutos(produtos.filter(p => p.id !== id))`, `setProduto({ ...produto, nome: "X" })`.
- O `set` é **assíncrono**: logo depois de `setContador(1)`, `contador` ainda tem o valor antigo naquela execução. Quando o novo depende do anterior: `setContador(c => c + 1)`.
- **Hooks** (`useState`, `useEffect`...) só podem ser chamados no **topo** do componente: nunca dentro de `if`, laço ou função interna.

Eventos

```tsx
<button onClick={remover}>          // passa a FUNÇÃO (correto)
<button onClick={() => remover(id)}> // precisa de parâmetro: arrow function
<button onClick={remover(id)}>       // ERRADO: executa na hora de renderizar!
<input onChange={(e) => setNome(e.target.value)} />  // e.target.value = texto digitado (sempre string)
<form onSubmit={enviar}>  ...  function enviar(e: React.FormEvent) { e.preventDefault(); ... }
```

6. Listas, keys e renderização condicional

```tsx
<ul>
  {produtos.map((produto) => (
    <li key={produto.id}>{produto.nome} - R$ {produto.valor}</li>   // key única e estável
  ))}
</ul>

{carregando ? <p>Carregando...</p> : <Tabela />}            // if/else
{erro && <p className="erro">{erro}</p>}                      // só se erro for "verdadeiro"
{produtos.length === 0 && <p>Nenhum produto</p>}
```

key ajuda o React a saber qual item mudou, entrou ou saiu. Deve ser **única entre irmãos** e **estável**: use o `id`, e o índice do `map` só em último caso. Sem key, o console mostra o aviso *"Each child in a list should have a unique key prop"*.

Métodos de array mais usados: `map` (transforma), `filter` (filtra), `find` (primeiro que satisfaz, como o `FirstOrDefault`), `some` (como o `Any`), `reduce` (acumula), `sort`. Spread: `[...lista, novo]`, `{...obj, campo: x}`.

7. useEffect e consumo da API

useEffect executa **efeitos colaterais**, ou seja, o que não é desenhar a tela: buscar dados na API, timers, título da página. O 2º argumento é o **array de dependências**:

Forma	Quando roda
`useEffect(() => {...}, [])`	**uma vez**, quando o componente aparece (montagem). É o caso típico de "carregar a lista"
`useEffect(() => {...}, [id])`	na montagem e sempre que `id` mudar
`useEffect(() => {...})`	depois de **toda** renderização (cuidado: com `set` dentro vira laço infinito)
`return () => {...}` dentro do efeito	limpeza (cleanup), ao desmontar. Ex.: `clearInterval`

```
// services/api.ts
import axios from "axios";
const api = axios.create({ baseURL: "http://localhost:5195" });
export default api;

// ProdutoListar.tsx
const [produtos, setProdutos] = useState<Produto[]>([]);
useEffect(() => { carregar(); }, []);

async function carregar() {
  try {
    const resposta = await api.get<Produto[]>("/api/produto/listar");
    setProdutos(resposta.data);                      // axios já converte o JSON: .data
  } catch (erro: any) {
    setMensagem(erro.response?.data);               // texto que a API mandou (ex.: 400/404)
  }
}
```

axios × fetch

`axios.get(url)` → `resposta.data` já é o objeto, e status 4xx/5xx **cai no catch**. `fetch(url)` é nativo: precisa de `await resposta.json()`, e 4xx/5xx **não** geram exceção (é preciso checar `resposta.ok`). POST com fetch: `fetch(url, { method: "POST", headers: { "Content-Type": "application/json" }, body: JSON.stringify(produto) })`.

Promise e async/await

Chamada HTTP é **assíncrona**: devolve uma **Promise** (um valor que chega depois). `await` espera a Promise e só pode ser usado dentro de função `async`. Alternativa: `.then(r => ...).catch(e => ...)`. O `useEffect` não pode ser `async` diretamente, então chame uma função async dentro dele.

CORS

React em `localhost:5173` e API em `localhost:5195` são **origens diferentes** (muda a porta). O **navegador** bloqueia a resposta, a menos que a API libere:

```
builder.Services.AddCors(options => options.AddPolicy("Acesso Total",
    configs => configs.AllowAnyOrigin().AllowAnyHeader().AllowAnyMethod()));
...
app.UseCors("Acesso Total");   // antes do app.Run()
```

Sintoma: *"has been blocked by CORS policy"* no console do navegador, enquanto no Postman/REST Client tudo funciona (eles não são navegadores).

8. CRUD completo e rotas

Ação na tela	Chamada (axios)	Endpoint C#
Listar ao abrir	`api.get("/api/produto/listar")` no `useEffect`	`MapGet` → 200 / 400 se vazia
Cadastrar (form)	`api.post("/api/produto/cadastrar", produto)`	`MapPost` → 201 / 400
Carregar p/ editar	`api.get(`/api/produto/buscarid/${id}`)`	`MapGet` + `Find` → 200 / 404
Salvar edição	`api.put(`/api/produto/alterar/${id}`, produto)`	`MapPut` → 200 / 404
Remover (botão)	`api.delete(`/api/produto/remover/${id}`)` e recarregar	`MapDelete` → 200 / 404

React Router (navegação na SPA)

```
import { BrowserRouter, Routes, Route, Link, useParams, useNavigate } from "react-router-dom";

<BrowserRouter>
  <nav><Link to="/">Listar</Link> <Link to="/produto/cadastrar">Cadastrar</Link></nav>
  <Routes>
    <Route path="/" element={<ProdutoListar />} />
    <Route path="/produto/cadastrar" element={<ProdutoCadastrar />} />
    <Route path="/produto/alterar/:id" element={<ProdutoAlterar />} />   // :id = parâmetro
  </Routes>
</BrowserRouter>

// dentro de ProdutoAlterar:
const { id } = useParams();        // lê o :id da URL
const navigate = useNavigate();    navigate("/");   // redireciona por código
```

Use `<Link>` e não `<a href>`: o `a` recarrega a página inteira e perde o estado, ou seja, deixa de ser SPA.

9. Formulários e validação

```tsx
function ProdutoCadastrar() {
  const [nome, setNome] = useState("");
  const [valor, setValor] = useState("");
  const [erros, setErros] = useState<Record<string, string>>({});

  async function enviar(e: React.FormEvent) {
    e.preventDefault();                                  // impede o reload da página
    const novos: Record<string, string> = {};
    if (nome.trim().length < 3) novos.nome = "Mínimo 3 caracteres";
    if (!(parseFloat(valor) > 0)) novos.valor = "Valor deve ser maior que zero";
    setErros(novos);
    if (Object.keys(novos).length > 0) return;          // tem erro: não envia
    try {
      await api.post("/api/produto/cadastrar", { nome, valor: parseFloat(valor) });
      navigate("/");
    } catch (erro: any) { setMensagem(erro.response?.data); }  // ex.: "Esse produto ja existe!"
  }
  return (
    <form onSubmit={enviar}>
      <input value={nome} onChange={e => setNome(e.target.value)} />
      {erros.nome && <span className="erro">{erros.nome}</span>}
      <input type="number" value={valor} onChange={e => setValor(e.target.value)} />
      {erros.valor && <span className="erro">{erros.valor}</span>}
      <button type="submit">Cadastrar</button>
    </form>
  );
}
```

Validar nos dois lados

Front: resposta imediata e menos requisições inúteis (usabilidade). **Back**: **obrigatório**, porque o front pode ser burlado (Postman, DevTools). A regra de negócio "nome único" só o back sabe verificar (consulta o banco), e o front exibe a mensagem que veio no 400.

Validação nativa do HTML

`required`, `minLength`, `maxLength`, `min`, `max`, `pattern`, `type="email"` / `"number"`. O navegador bloqueia o submit e mostra um balão. `noValidate` no `form` desliga isso, para usar a validação própria. Bibliotecas: React Hook Form, Formik, Yup/Zod.

10. Layout responsivo

- **Responsivo**: o layout se adapta à largura da tela (celular, tablet, desktop). Precisa da meta tag `<meta name="viewport" content="width=device-width, initial-scale=1.0">` (o Vite já inclui).
- **Flexbox** (uma dimensão): `display: flex; flex-direction: row|column; justify-content; align-items; gap; flex-wrap: wrap`.
- **CSS Grid** (duas dimensões): `display: grid; grid-template-columns: repeat(3, 1fr); gap: 16px`. `repeat(auto-fit, minmax(200px, 1fr))` ajusta sozinho o número de colunas.
- **Media queries**: `@media (max-width: 600px) { ... }` aplica regras só em telas pequenas. *Mobile first*: escreve para celular e usa `min-width` para telas maiores.
- **Frameworks de grid**: **Bootstrap** (grid de 12 colunas: `container` → `row` → `col-12 col-md-6 col-lg-4`, breakpoints sm/md/lg/xl), **Tailwind** (classes utilitárias: `grid grid-cols-1 md:grid-cols-3`), MUI.

- Unidades: `px` (fixa), `%`, `rem` (relativa à fonte da raiz), `vw/vh` (largura/altura da tela), `fr` (fração no grid).
- **Transições e animações** (Plano de Ensino): `transition: all 0.3s` em `:hover`, e `@keyframes` + `animation`.

```
.grade { display: grid; grid-template-columns: repeat(3, 1fr); gap: 16px; }
@media (max-width: 768px) { .grade { grid-template-columns: 1fr; } }   /* 1 coluna no celular */
.botao { transition: background 0.3s; }  .botao:hover { background: #1d4e89; }
```

11. Pegadinhas

1. `onClick={remover(id)}` executa na renderização. O certo é `onClick={() => remover(id)}`.
2. Alterar o estado direto (`lista.push`) não re-renderiza. Use o `set` com um novo array ou objeto.
3. `useEffect` sem `[]` e com `set` dentro é laço infinito de requisições.
4. Faltou `key` no `map`: aparece um aviso e a atualização da lista pode sair errada.
5. `class` no JSX é `className`. Componente com letra minúscula não é reconhecido como componente.
6. Esquecer `e.preventDefault()` no submit: a página recarrega e perde tudo.
7. `e.target.value` é sempre **string**. Converta com `parseFloat`/`Number` antes de enviar.
8. Erro de CORS é bloqueio do **navegador**, e a correção é na **API** (`AddCors` + `UseCors`).
9. Os nomes das propriedades no JSON são **camelCase** (`nome`, `valor`), não `Nome`.
10. `<a href>` recarrega a página. Na SPA use `<Link to>`.
11. Com axios, os status 400/404 caem no `catch`. A mensagem da API está em `erro.response.data`.
12. `await` só dentro de `async`. O callback do `useEffect` não pode ser `async`.
13. Rodar o React sem a API ligada dá *Network Error*. Suba primeiro a API (`dotnet run`) e depois o front (`npm run dev`).

12. Revisão relâmpago da API (conteúdo da A2)

Tema	Essencial
Verbos	GET ler · POST criar · PUT substituir · PATCH alterar parte · DELETE remover
Status	200 OK · 201 Created · 204 No Content · 400 Bad Request · 404 Not Found · 409 Conflict · 500 Server Error
Minimal API	`builder` → `builder.Services.Add...` → `Build()` → `app.MapGet/MapPost/MapPut/MapDelete` → `app.Run()`
Parâmetros	`[FromRoute] {id}` · `[FromQuery] ?x=1` · `[FromBody]` JSON · `[FromServices]` DbContext
EF Core (5 passos)	pacotes (Sqlite + Design) → classe `AppDataContext` → `: DbContext` → `DbSet<Produto>` → `OnConfiguring` com `UseSqlite("Data Source=Ecommerce.db")`; depois `AddDbContext`
Migrations	`dotnet ef migrations add Nome` → `dotnet ef database update`. Mudou o modelo (ex.: `Valor`) → nova migration
CRUD EF	`ToList()`, `Find(id)`, `FirstOrDefault(x => ...)`, `Add` / `Update` / `Remove` + **`SaveChanges()`**
POO	classe/objeto, propriedades get/set, construtor, encapsulamento, herança `:`, polimorfismo `virtual/override`, interface
Outros do plano	**Swagger**: documentação interativa da API (pacote Swashbuckle, `AddSwaggerGen` / `UseSwagger` / `UseSwaggerUI`, página `/swagger`). **JWT**: token assinado (header.payload.signature) que o cliente manda em `Authorization: Bearer <token>`. **Autenticação** = quem é você (401). **Autorização** = o que pode fazer (403)

13. Questões com gabarito

1. O que é uma SPA e qual a vantagem?

Aplicação de página única: carrega um HTML e o JavaScript troca o conteúdo sem recarregar a página, buscando dados em JSON na API. Navegação mais rápida e fluida, com front e back separados.

2. Diferença entre props e estado?

Props vêm do componente pai e são somente leitura. Estado é interno do componente, muda com o `set` e provoca nova renderização.

3. Por que este código não atualiza a tela? `let total = 0; ... <button onClick={() => total++}>`

Variável comum não dispara re-renderização. É preciso `const [total, setTotal] = useState(0)` e `setTotal(total + 1)`.

4. Quando roda `useEffect(() => { carregar(); }, [])`?

Uma única vez, quando o componente é montado (aparece na tela). É o lugar de buscar a lista na API.

5. A API funciona no Postman, mas o React mostra "blocked by CORS policy". Por quê e como resolver?

Front e API estão em origens diferentes (portas 5173 e 5195), e o navegador bloqueia sem autorização da API. Resolver na API:
`builder.Services.AddCors(...AllowAnyOrigin().AllowAnyHeader().AllowAnyMethod())` e `app.UseCors("...")`.

6. Escreva o código que remove um produto e atualiza a lista sem buscar de novo na API.

```
async function remover(id: string) {
  await api.delete(`/api/produto/remover/${id}`);
  setProdutos(produtos.filter(p => p.id !== id));
}
```

7. Para que serve a `key` em `produtos.map(p => <li key={p.id}>...)`?

Identificar cada item de forma única e estável, para o React saber o que foi adicionado, removido ou alterado e atualizar só aquele elemento.

8. Por que validar no front se a API já valida?

Pela experiência do usuário: o retorno é imediato e evita requisições desnecessárias. A validação do back continua obrigatória, porque o front pode ser burlado.

9. Como a tela de alterar descobre qual produto editar?

A rota tem um parâmetro (`/produto/alterar/:id`). `useParams()` lê o `id` , o `useEffect([id])` faz `GET buscarid/{id}` e preenche o estado do formulário. O salvar faz `PUT alterar/{id}` .

10. Complete: `const [produtos, setProdutos] = _____<Produto[]>(___);`

`useState` e `[]` (começa com uma lista vazia).

11. Qual comando baixa as dependências de um projeto React clonado do Git, e por que é necessário?

`npm install` . A pasta `node_modules` não vai para o Git, e o comando a recria a partir do `package.json` e do `package-lock.json` .

12. Encontre 3 erros:

```
function lista() {
  const [itens, setItens] = useState([]);
  useEffect(async () => { const r = await api.get("/api/produto/listar"); itens = r.data; });
  return <ul class="lista">{itens.map(i => <li>{i.nome}</li>)}</ul>;
}
```

(1) O componente deve começar com maiúscula (`Lista`). (2) O `useEffect` não pode ser async e está sem `[]` (roda sempre). (3) `itens = r.data` altera o estado direto, então deve ser `setItens(r.data)` . Também: `class` deve ser `className` e falta a `key` no `li` .

Material complementar de estudo. O projeto `08-React-Ecommerce` do repositório de estudos tem todo este código funcionando e testado no navegador.